>_ MCMP113 · Week 14

Basic File Operations · Semester Review

Text files and paths, the File class, StreamWriter and StreamReader, CSV data, exceptions, a review of the semester and the Student Grade Manager mini-project

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

These notes follow the Week 14 slides. Part 1 (Sections 2–7) teaches text files; Part 2 (Section 8) reviews the whole semester in one table per topic; Part 3 (Section 9) walks through the mini-project *Student Grade Manager*. Every listing is read from the tested programs in the `code` folder (`14_01` ... `14_12`); open the folder in VS Code and run it with `dotnet run` while you read. All programs create the files they read themselves, so they run on every computer. All names and grades are invented. Typed input in the sample runs is shown in amber.

Contents

1 Learning outcomes 2
2 Why files? Text files and paths 2
3 Streams: line by line 4
4 CSV files 6
5 Keeping data between runs 8
6 Reading and transforming text 11
7 When things go wrong: exceptions 13
8 Semester review 15
9 Mini-project: Student Grade Manager 16
10 Common mistakes 21
11 Summary 22
12 Glossary (English – Turkish) 23
13 Exercises 23
14 References 24

>_ 1 Learning outcomes

After this week you should be able to:

1. explain why programs need files, distinguish text from binary files, and predict where a relative path points (the working directory);
2. write, append and read whole files with `File.WriteAllText`, `AppendAllText`, `ReadAllText`, `WriteAllLines` and `ReadAllLines`, and use `File.Exists`, `Delete`, `Copy` and `Path.Combine`;
3. write and read files line by line with `StreamWriter` and `StreamReader` inside `using`, reading until `ReadLine()` returns `null`;
4. process CSV data: split a line with `Split(',')`, parse the fields and compute totals per row;
5. keep data between runs (to-do list, log file, high score, account balance);
6. handle `FileNotFoundException`, `IOException` and `FormatException` with `try/catch` in the right order;
7. connect the topics of Weeks 1–14 in one program: menu, arrays, methods and files.

>_ 2 Why files? Text files and paths

2.1 Data that outlives the program

Variables and arrays live in the main memory (RAM). When a program ends, the operating system takes this memory back and every value is lost: a to-do list typed yesterday, the best score of a game, a bank balance. A **file** lives on the disk until somebody deletes it. Saving means copying data from variables into a file; loading means copying it back at the next start.

	Text file	Binary file
Contains	characters, stored as UTF-8	raw bytes in a program-specific format
Open with	any editor: Notepad, VS Code	only the program that understands it
Examples	`.txt`, `.csv`, `.log`, `.cs`	`.exe`, `.dll`, `.png`, `.docx`
In MCMP113	Weeks 14–15	not used

.NET writes and reads text files as **UTF-8** by default, so Turkish letters (ğ, ü, ş, ı, ö, ç) are stored correctly. Only the Windows console may need `Console.OutputEncoding = System.Text.Encoding.UTF8;` to *display* them (Example 7).

2.2 Relative paths and the working directory

A **path** names a file. `"notes.txt"` is a **relative path**: it is looked up in the **working directory** (current directory) of the running program. With `dotnet run` — and with *Run* in VS Code — the working directory is the **project folder**, so the file appears next to `Program.cs`. The compiled program itself is in `bin/Debug/net10.0`; this folder is `AppContext.BaseDirectory`. If you start the `.dll` from a different folder, relative paths point to *that* folder and the program will not find its files.

An **absolute path** such as `C:\Users\ali\Desktop\notes.txt` starts at the root of a drive. Avoid it in programs: it works only on one computer. To build a path from a folder and a file name use `Path.Combine("backup", "notes_copy.txt");` it inserts the right separator, `/` on Linux and macOS, `\` on Windows. Everything we need is in the namespace `System.IO`, which our project template imports automatically (`ImplicitUsings`).

2.3 The File class

The static methods of `File` open a file, do their job and close it again — in one call. They are ideal for small files.

Method	What it does	Returns
`File.WriteAllText(path, text)`	creates the file, or **replaces** its whole content	`--`
`File.AppendAllText(path, text)`	adds text at the end (creates the file if missing)	`--`
`File.ReadAllText(path)`	reads the whole file	`string`
`File.WriteAllLines(path, lines)`	writes one array element per line	`--`
`File.ReadAllLines(path)`	reads every line into an array	`string[]`
`File.AppendAllLines(path, lines)`	adds lines at the end	`--`
`File.Exists(path)`	is the file there?	`bool`
`File.Delete(path)`	deletes it (no error if it is missing)	`--`
`File.Copy(from, to, true)`	copies; `true` = overwrite the target	`--`

2.4 Example 1 – first steps with text files

The program prints its working directory, creates `notes.txt`, appends a line, reads everything back, saves a copy into the sub-folder `backup`, and finally shows the classic mistake: a second `WriteAllText` *replaces* the content. The 56 characters of step 4 include the three line breaks `\n`.

Listing 1: 14_01_FirstFile/Program.cs

```
// 1. A relative path starts in the WORKING DIRECTORY.
//    With `dotnet run` (or Run in VS Code) this is the project folder.
string workDir = Directory.GetCurrentDirectory();
Console.WriteLine($"Working directory: {Path.GetFileName(workDir)}");
string exeDir = Path.GetRelativePath(workDir, AppContext.BaseDirectory);
Console.WriteLine($"Program files in : {exeDir}");

// 2. Create the file (or replace its old content) with one call
string path = "notes.txt";
File.WriteAllText(path, "MCMP113 notes\nWeek 14: files\n");
Console.WriteLine($"Created? {File.Exists(path)}");

// 3. Add text to the END of the file
File.AppendAllText(path, "Data outlives the program.\n");

// 4. Read the whole file back as ONE string
string content = File.ReadAllText(path);
Console.WriteLine($"{path}: {content.Length} characters");
Console.Write(content);
Console.WriteLine("---");

// 5. Path.Combine puts the right separator between folder and file name
Directory.CreateDirectory("backup");              // no error if it already exists
string copyPath = Path.Combine("backup", "notes_copy.txt");
File.WriteAllText(copyPath, content);
Console.WriteLine($"Copy saved as {copyPath}");   // backup\notes_copy.txt on Windows

// 6. WriteAllText again: the old content is REPLACED, not extended
File.WriteAllText(path, "Only this line is left.\n");
Console.Write($"Now: {File.ReadAllText(path)}");

// 7. Delete the file (no error if it does not exist)
File.Delete(path);
Console.WriteLine($"Deleted? {!File.Exists(path)}");
Console.WriteLine($"Copy kept? {File.Exists(copyPath)}");
```

```
Sample run (Linux: forward slashes; on Windows backup\notes_copy.txt)

Working directory: 14_01_FirstFile
Program files in : bin/Debug/net10.0/
Created? True
notes.txt: 56 characters
MCMP113 notes
Week 14: files
Data outlives the program.
---
Copy saved as backup/notes_copy.txt
Now: Only this line is left.
Deleted? True
Copy kept? True
```

2.5 Example 2 – lines and arrays

`WriteAllLines` and `ReadAllLines` connect files with the arrays of Week 9: one element is one line. The numbers are converted to strings for saving and back with `double.Parse` for processing; `loaded.Length` tells how many lines the file has, so the program does not need to know it in advance. Note that `16.0.ToString()` gives `"16"`.

Listing 2: 14_02_LinesAndArrays/Program.cs (lines 6–21)

```
6  // 1. Save a week of temperatures: number array -> string array -> file
7  double[] week = { 14.5, 16.0, 13.5, 18.0, 19.5, 17.0, 15.5 };
8  string[] lines = new string[week.Length];
9  for (int i = 0; i < week.Length; i++)
10 {
11     lines[i] = week[i].ToString();                // 14.5 -> "14.5"
12 }
13 File.WriteAllLines(path, lines);
14 Console.WriteLine($"Saved {lines.Length} lines to {path}");
15
16 // 2. AppendAllLines adds lines at the end (here: one more day)
17 File.AppendAllLines(path, new string[] { "21.0" });
18
19 // 3. Load: the file becomes a string[], one element per line
20 string[] loaded = File.ReadAllLines(path);
21 Console.WriteLine($"Loaded {loaded.Length} lines: {string.Join(" | ", loaded)}");
```

```
Sample run

Saved 7 lines to temperatures.txt
Loaded 8 lines: 14.5 | 16 | 13.5 | 18 | 19.5 | 17 | 15.5 | 21.0
Average: 16.88 C
Warmest: day 8 with 21.0 C
```

Check: $\bar{t} = (14.5 + 16 + 13.5 + 18 + 19.5 + 17 + 15.5 + 21)/8 = 135/8 = 16.875 \approx 16.88$.

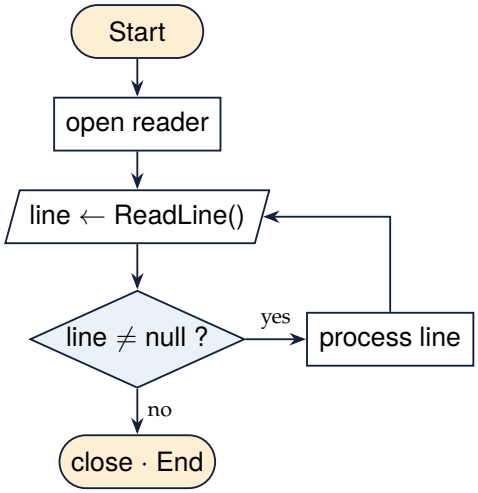
>_ 3 Streams: line by line

For many lines written in a loop, or for very large files, we keep the file *open* with a **stream**. A `StreamWriter` writes, a `StreamReader` reads. Both must be closed when we are done — otherwise the last lines may stay in a memory buffer and never reach the disk. The **using statement** closes the stream automatically at the end of its block, even if an exception happens:

```
using (StreamWriter writer = new StreamWriter(path))        // new file (old content replaced)
{
    writer.WriteLine("one line");
```

```
}                                                       // file closed here
using (StreamWriter writer = new StreamWriter(path, true))   // true: APPEND mode
{
    writer.WriteLine("one more line");
}
```

`ReadLine()` returns the next line, or `null` when there are no more lines. Reading a file is therefore the sentinel-controlled loop of Week 7 with the sentinel `null`:



Read a file line by line

START
open the file
READ *line*
WHILE *line* ≠ null
    process *line*
    READ *line*
END WHILE
close the file
END

In C# the same loop is written in two common forms. Form A (Example 3) reads once before the loop and once at the end of the body; Form B (Example 9) reads and tests in the condition, because an assignment has a value:

Form A

```
string line = reader.ReadLine();
while (line != null)
{
    // process line
    line = reader.ReadLine();
}
```

Form B

```
string line;
while ((line = reader.ReadLine()) != null)
{
    // process line
}
```

Every call of `ReadLine()` consumes one line. Calling it twice in one pass skips every second line (Exercise E4).

3.1 Example 3 – StreamWriter and StreamReader

Listing 3: 14_03_StreamReadWrite/Program.cs (lines 4–31)

```
4  Console.Write("Multiplication table for n = ");
5  int n = int.Parse(Console.ReadLine());
6  string path = $"table_{n}.txt";
7
8  // using: the file is closed automatically at the end of the block
9  using (StreamWriter writer = new StreamWriter(path))
10 {
11     writer.WriteLine($"Multiplication table for {n}");
12     for (int i = 1; i <= 10; i++)
13     {
14         writer.WriteLine($"{n} x {i,2} = {n * i,3}");
15     }
16 }
```

```
Console.WriteLine($"Saved {path}");

// Read line by line: ReadLine() returns null at the end of the file
int lineNo = 0;
using (StreamReader reader = new StreamReader(path))
{
    string line = reader.ReadLine();
    while (line != null)
    {
        lineNo++;
        Console.WriteLine($"{lineNo,2}| {line}");
        line = reader.ReadLine();
    }
}
Console.WriteLine($"{lineNo} lines read");
```

Sample run (the last line comes from the append part at the end of the program)

```
Multiplication table for n = 7
Saved table_7.txt
 1| Multiplication table for 7
 2| 7 x  1 =  7
 3| 7 x  2 =  14
 4| 7 x  3 =  21
 5| 7 x  4 =  28
 6| 7 x  5 =  35
 7| 7 x  6 =  42
 8| 7 x  7 =  49
 9| 7 x  8 =  56
10| 7 x  9 =  63
11| 7 x 10 =  70
11 lines read
Now 12 lines, the last one is: -- end of table --
```

>_ 4 CSV files

CSV (comma-separated values) is the simplest table format: one *record* per line, *fields* separated by commas, often a header line first. Excel and Google Sheets open CSV files directly. `Split(',')` cuts a line into a `string[]`; numeric fields are still text and must be parsed:

Expression	Value
`lines[1]`	`"Ayse Kaya,72,64"`
`lines[1].Split(',')`	`{ "Ayse Kaya", "72", "64" }  (Length 3)`
`int.Parse(fields[1])`	`72`
`0.4 * 72 + 0.6 * 64`	`67.2` (MCMP113 weights: midterm 40%, final 60%)

Three rules keep our CSV files simple: **(1)** skip the header line (start the loop at $i = 1$; `int.Parse("midterm")` would throw a `FormatException`); **(2)** values must not contain commas — a comma inside a name creates an extra field (real CSV allows quoted values, which needs a CSV library); **(3)** numbers use a decimal *point* (`72.5`); our template's `InvariantGlobalization` guarantees that.

Gradebook CSV → report

START
lines ← all lines of grades.csv
FOR $i \leftarrow 1$ **TO** $lines.Length - 1$ ▷ line 0 is the header
    $fields \leftarrow$ split $lines[i]$ at the commas
    $avg \leftarrow 0.4 \cdot fields[1] + 0.6 \cdot fields[2]$
    write $fields[0]$, avg and the letter grade of avg to report.txt
END FOR
END

4.1 Example 4 – gradebook CSV to report file

The program first writes `grades.csv` itself (synthetic data), then reads it and writes `report.txt` with a `StreamWriter` while it walks through the rows. `LetterGrade` is the Week 5 example scale (check the university regulation for the official cut-offs).

Listing 4: 14_04_GradebookCsv/Program.cs (lines 19–47; LetterGrade as in Example 12)

```
19 // 2. Read the CSV and write the report at the same time
20 string[] lines = File.ReadAllLines(csvPath);
21 double classSum = 0;
22 int passed = 0;
23 using (StreamWriter report = new StreamWriter(reportPath))
24 {
25     report.WriteLine($"{"Student",-14}{"Mid",4}{"Fin",5}{"Avg",7}  Grade");
26     for (int i = 1; i < lines.Length; i++)              // i = 1: skip the header row
27     {
28         string[] fields = lines[i].Split(',');          // "Ayse Kaya,72,64" -> 3 fields
29         string name = fields[0];
30         int midterm = int.Parse(fields[1]);
31         int final = int.Parse(fields[2]);
32         double average = 0.4 * midterm + 0.6 * final;   // MCMP113 weights: 40 % + 60 %
33         string letter = LetterGrade(average);
34         report.WriteLine($"{name,-14}{midterm,4}{final,5}{average,7:F1}  {letter}");
35         classSum += average;
36         if (letter != "FF")
37         {
38             passed++;
39         }
40     }
41     int students = lines.Length - 1;
42     report.WriteLine($"Class average: {classSum / students:F2}, passed: {passed} of {students}");
43 }
44
45 // 3. Show the report that was written
46 Console.WriteLine($"{reportPath} written. Its content:");
47 Console.Write(File.ReadAllText(reportPath));
```

Sample run

```
report.txt written. Its content:
Student       Mid  Fin   Avg  Grade
Ayse Kaya      72   64  67.2  DC
Burak Demir    55   66  61.6  DD
Ceren Yildiz   88   95  92.2  AA
Deniz Arslan   58   61  59.8  FF
Emre Sahin     30   41  36.6  FF
Class average: 63.48, passed: 3 of 5
```

Check of the last line: $(67.2 + 61.6 + 92.2 + 59.8 + 36.6)/5 = 317.4/5 = 63.48$. Deniz Arslan (59.8)

is a boundary case just below 60.

>_ 5 Keeping data between runs

Four small programs keep their data in a file so that the next run can continue. The pattern is always the same: **load** at the start (with a sensible default when the file does not exist yet), **save** after every change (or at the end). Each program works on its first run — delete its data file to start again.

5.1 Example 5 – a to-do list

New tasks are appended with `AppendAllText`; `Environment.NewLine` is the line break of the operating system (`\r\n` on Windows, `\n` elsewhere). A finished task cannot be removed "in the middle of a file": the program loads all lines, builds a new array without that element (`RemoveAt`, returning an array as in Week 13) and rewrites the whole file.

Listing 5: 14_05_ToDoList/Program.cs (lines 23–29)

```
23      else if (choice == "2")
24      {
25          Console.Write("New task: ");
26          string task = Console.ReadLine();
27          File.AppendAllText(path, task + Environment.NewLine);   // one new line at the end
28          Console.WriteLine("Saved.");
29      }
```

```
49  // Returns all tasks from the file, or an empty array if the file does not exist yet
50  static string[] LoadTasks(string path)
51  {
52      if (!File.Exists(path))
53      {
54          return new string[0];
55      }
56      return File.ReadAllLines(path);
57  }
```

Run 1 (end)

```
Saved.
1 List  2 Add  3 Done  0 Quit > 1
  1. Revise arrays for the final
  2. Finish lab 14 exercises
  3. Email the project group
1 List  2 Add  3 Done  0 Quit > 0
Bye! Your list is saved.
```

Run 2 (start): the list was loaded

```
Welcome back! 3 task(s) loaded from todo.txt.
1 List  2 Add  3 Done  0 Quit > 1
  1. Revise arrays for the final
  2. Finish lab 14 exercises
  3. Email the project group
1 List  2 Add  3 Done  0 Quit > 3
  1. Revise arrays for the final
  2. Finish lab 14 exercises
  3. Email the project group
Number of the finished task: 2
```

5.2 Example 6 – a log file with timestamps

A log only grows: every event is appended as one line with the date and time. In the format `yyyy-MM-dd HH:mm`, `MM` is the month, `mm` the minutes and `HH` the hour on the 24-hour clock. `Log` is a `void` method: its job is an action.

Listing 6: 14_06_LogFile/Program.cs (lines 32–37: the method Log)

```
32  // Appends one line with the current date and time to the log file
33  static void Log(string path, string level, string message)
```

```
{
    string line = $"{DateTime.Now:yyyy-MM-dd HH:mm} | {level} | {message}";
    File.AppendAllText(path, line + Environment.NewLine);
}
```

Second run (input 1 and 19.5): the first five log lines come from run 1 (input 3, 22.5, 31.2, 27)

```
How many readings? 1
Temperature 1: 19.5
--- sensor.log: 8 lines ---
2026-10-12 21:25 | INFO | Program started
2026-10-12 21:25 | INFO | Reading 1: 22.5 C
2026-10-12 21:25 | WARN | Reading 2: 31.2 C is too high
2026-10-12 21:25 | INFO | Reading 3: 27.0 C
2026-10-12 21:25 | INFO | Program finished
2026-10-12 21:25 | INFO | Program started
2026-10-12 21:25 | INFO | Reading 1: 19.5 C
2026-10-12 21:25 | INFO | Program finished
```

5.3 Example 8 – a high-score file for the guessing game

The Week 7 guessing game keeps its record in `highscore.txt` as one line `name,attempts`. `LoadHighScore` returns a tuple (Week 13); a damaged file is ignored. The secret is random (`Random.Shared.Next(1, 101)`), so every run is different; the runs below were played with the halving strategy of Week 1, which needs at most 7 guesses because $2^7 = 128 \geq 100$.

Listing 7: 14_08_HighScore/Program.cs (lines 36–67)

```
if (bestAttempts == 0 || attempts < bestAttempts)
{
    File.WriteAllText(path, $"{name},{attempts}");      // one line: name,attempts
    Console.WriteLine("New record saved!");
}
else
{
    Console.WriteLine($"The record stays with {bestName} ({bestAttempts}).");
}

// Reads "name,attempts" from the file; returns ("", 0) if there is no valid record
static (string name, int attempts) LoadHighScore(string path)
{
    if (!File.Exists(path))
    {
        return ("", 0);
    }
    try
    {
        string[] parts = File.ReadAllText(path).Split(',');
        if (parts.Length != 2)
        {
            return ("", 0);
        }
        return (parts[0], int.Parse(parts[1]));
    }
    catch (FormatException)
    {
        Console.WriteLine("(damaged high-score file ignored)");
        return ("", 0);
    }
}
```

Run 2: a new record

```
Record to beat: 7 attempts by Ayla
Your name: Deniz
Guess 1: 50
Too small.
Guess 2: 75
Too big.
Guess 3: 62
Too small.
Guess 4: 68
Too small.
Guess 5: 71
Too big.
Guess 6: 69
Correct, Deniz! 6 attempts.
New record saved!
```

Run 3: the record stays

```
Record to beat: 6 attempts by Deniz
Your name: Mert
Guess 1: 50
Too big.
Guess 2: 25
Too big.
Guess 3: 12
Too big.
Guess 4: 6
Too small.
Guess 5: 9
Too small.
Guess 6: 10
Too small.
Guess 7: 11
Correct, Mert! 7 attempts.
The record stays with Deniz (6).
```

5.4 Example 10 – a bank account

`balance.txt` holds one `decimal` written with `F2`; it is saved after every transaction, and `history.csv` receives one appended line per transaction. Run 3 shows a damaged file: we typed the Turkish notation `1.550,50` into `balance.txt` by hand; `decimal.Parse` throws a `FormatException`, the program warns and uses 0. A program must read exactly the format it writes — and a real banking program would stop and ask instead of using 0.

Listing 8: 14_10_BankAccount/Program.cs (lines 46–68: LoadBalance and SaveBalance)

```
46 // Reads the balance from the file; 0 if the file is missing or damaged
47 static decimal LoadBalance(string path)
48 {
49     if (!File.Exists(path))
50     {
51         return 0;
52     }
53     try
54     {
55         return decimal.Parse(File.ReadAllText(path));
56     }
57     catch (FormatException)
58     {
59         Console.WriteLine($"Warning: {path} damaged; using 0");
60         return 0;
61     }
62 }
63
64 // Writes the balance into the file (old content replaced)
65 static void SaveBalance(string path, decimal balance)
66 {
67     File.WriteAllText(path, balance.ToString("F2"));
68 }
```

Run 1

```
Balance loaded: 0.00 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 1
Amount to deposit: 1500
New balance: 1500.00 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 2
Amount to withdraw: 5000
Insufficient balance.
1 Deposit  2 Withdraw  3 History  0 Exit > 2
Amount to withdraw: 200
New balance: 1300.00 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 3
2026-10-12 21:29,deposit,1500.00,1500.00
2026-10-12 21:29,withdraw,200.00,1300.00
1 Deposit  2 Withdraw  3 History  0 Exit > 0
Goodbye. Final balance: 1300.00 TL
```

Run 2

```
Balance loaded: 1300.00 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 1
Amount to deposit: 250.50
New balance: 1550.50 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 3
2026-10-12 21:29,deposit,1500.00,1500.00
2026-10-12 21:29,withdraw,200.00,1300.00
2026-10-12 21:29,deposit,250.50,1550.50
1 Deposit  2 Withdraw  3 History  0 Exit > 0
Goodbye. Final balance: 1550.50 TL
```

Run 3 (damaged balance.txt)

```
Warning: balance.txt damaged; using 0
Balance loaded: 0.00 TL
1 Deposit  2 Withdraw  3 History  0 Exit > 0
Goodbye. Final balance: 0.00 TL
```

>_ 6 Reading and transforming text

6.1 Example 7 – line, word and character counter

The line count is `lines.Length` (the empty line counts too). `Split(' ', StringSplitOptions.RemoveEmptyEntries)` ignores double spaces, so it counts words correctly; characters are the sum of the line lengths without line breaks; `char.IsLetter` also recognises Turkish letters.

Listing 9: 14_07_TextStats/Program.cs (lines 4–55)

```
4  Console.OutputEncoding = System.Text.Encoding.UTF8;    // shows ğ ü ş ı ö ç correctly on Windows
       too
5
6  string sample = "proverbs.txt";
7  if (!File.Exists(sample))
8  {
9      string[] text =
10     {
11         "Damlaya damlaya göl olur.",
12         "Ağaç yaş iken eğilir.",
13         "",
14         "Bugünün işini yarına bırakma.",
15         "Practice makes perfect.",
16     };
17     File.WriteAllLines(sample, text);                    // UTF-8 is the default encoding
18     Console.WriteLine($"Created {sample}");
19 }
20
21 Console.Write("File to analyse: ");
22 string path = Console.ReadLine();
```

```
try
{
    string[] lines = File.ReadAllLines(path);
    int words = 0, characters = 0, letters = 0, longest = 0;
    for (int i = 0; i < lines.Length; i++)
    {
        words += CountWords(lines[i]);
        characters += lines[i].Length;                    // line breaks are not counted
        letters += CountLetters(lines[i]);
        if (lines[i].Length > lines[longest].Length)
        {
            longest = i;                                  // index of the longest line so far
        }
    }
    Console.WriteLine($"Lines     : {lines.Length}");
    Console.WriteLine($"Words     : {words}");
    Console.WriteLine($"Characters: {characters}");
    Console.WriteLine($"Letters   : {letters}");
    if (lines.Length > 0)
    {
        Console.WriteLine($"Longest   : line {longest + 1}, \"{lines[longest]}\"");
    }
}
catch (FileNotFoundException)
{
    Console.WriteLine($"Sorry, there is no file called \"{path}\" in this folder.");
}

// Number of words: pieces between spaces, empty pieces ignored
static int CountWords(string line)
{
    return line.Split(' ', StringSplitOptions.RemoveEmptyEntries).Length;
}
```

Run 1

```
Created proverbs.txt
File to analyse: proverbs.txt
Lines     : 5
Words     : 15
Characters: 98
Letters   : 83
Longest   : line 4, "Bugünün işini yarına bırakma."
```

Run 2: typing error in the file name

```
File to analyse: proverb.txt
Sorry, there is no file called "proverb.txt" in this folder.
```

Check: characters $25+21+0+29+23=98$; words $4+4+0+4+3=15$; letters $98-11$ spaces $-$ 4 full stops $=83$.

6.2 Example 9 – copy and filter lines

Two `using` statements in a row open the source for reading and the target for writing. `Contains(keyword, StringComparison.OrdinalIgnoreCase)` ignores upper and lower case. `File.Copy` makes a full backup first.

Listing 10: 14_09_FilterCopy/Program.cs (lines 15–41)

```
File.Copy(source, "announcements_backup.txt", true);    // full copy; true = overwrite
Console.WriteLine("Backup copy created.");
```

```
17
18  Console.Write("Keyword: ");
19  string keyword = Console.ReadLine();
20  string target = $"filtered_{keyword}.txt";
21
22  int total = 0, copied = 0;
23  using (StreamReader reader = new StreamReader(source))
24  using (StreamWriter writer = new StreamWriter(target))
25  {
26      string line;
27      while ((line = reader.ReadLine()) != null)           // read, then test for the end
28      {
29          total++;
30          if (line.Contains(keyword, StringComparison.OrdinalIgnoreCase))
31          {
32              writer.WriteLine(line);
33              copied++;
34          }
35      }
36  }
37  Console.WriteLine($"{copied} of {total} lines copied to {target}:");
38  foreach (string line in File.ReadAllLines(target))
39  {
40      Console.WriteLine($"  {line}");
41  }
```

Sample run 1

```
Backup copy created.
Keyword: exam
2 of 5 lines copied to filtered_exam.txt:
  08.10 Final exam: date will be announced
  12.10 Exam rooms: see the course group
```

Sample run 2

```
Backup copy created.
Keyword: lab
2 of 5 lines copied to filtered_lab.txt:
  05.10 Lab 13 solutions are online
  09.10 Lab 14 is about text files
```

>_ 7 When things go wrong: exceptions

Files are outside our program: they can be missing, renamed, opened by Excel, or full of unexpected text. Then .NET **throws** an **exception**. If nobody catches it, the program stops with an error message (real output, path shortened):

Unhandled exceptions

```
Unhandled exception. System.IO.FileNotFoundException: Could not find file '.../MyProject/grades.txt'.
Unhandled exception. System.FormatException: The input string '70' was not in a correct format.
```

A `try/catch` statement handles the problem. If a statement inside `try` throws an exception of the type named in a `catch`, the *rest of the try block is skipped*, the `catch` block runs, and the program continues after the whole statement.

Exception	Typical cause
`FileNotFoundException`	the file does not exist (typing error, wrong working directory)
`IOException`	general input/output problem: missing folder (`DirectoryNotFoundException` is a kind of `IOException`), file locked by another program, disk full
`FormatException`	`int.Parse`, `double.Parse`, `decimal.Parse` of text that is not a number

Order: with several `catch` blocks the first matching one runs, so the specific type must come before the general one. `FileNotFoundException` is a kind of `IOException`; if `catch (IOException)` comes

first, the compiler refuses:

```
dotnet build

error CS0160: A previous catch clause already catches all exceptions of this or of a super type ('IOException')
```

Exceptions or TryParse? For keyboard input and for values we can test ourselves, `TryParse` is simpler and quieter (Weeks 5 and 12). Exceptions are for problems we cannot check in advance, such as a file that disappears or is locked. Never write an empty `catch { }`: a hidden error is worse than a crash.

7.1 Example 11 – safe reading

Part 2 catches the missing folder with the general `IOException`. In part 3 the `try`/`catch` is *inside* the loop, so one bad line only skips that line; `"70"` contains the letter O, not a zero.

Listing 11: 14_11_SafeReading/Program.cs

```
// 1. A file that does not exist
try
{
    string text = File.ReadAllText("no_such_file.txt");
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.WriteLine("1) file not found: no_such_file.txt");
}

// 2. A folder that does not exist: DirectoryNotFoundException is a kind of IOException
string oldPath = Path.Combine("old_data", "scores.txt");
try
{
    string text = File.ReadAllText(oldPath);
    Console.WriteLine(text);
}
catch (FileNotFoundException)
{
    Console.WriteLine("2) the file is missing");
}
catch (IOException)                    // general case: AFTER the specific one
{
    Console.WriteLine("2) IOException: no folder old_data");
}

// 3. Bad data inside a file: int.Parse throws FormatException
string path = "scores.txt";
File.WriteAllLines(path, new string[] { "78", "91", "abc", "65", "", "7O", "100" });
string[] lines = File.ReadAllLines(path);
int sum = 0, good = 0;
for (int i = 0; i < lines.Length; i++)
{
    try
    {
        sum += int.Parse(lines[i]);
        good++;
    }
    catch (FormatException)
    {
        Console.WriteLine($"3) line {i + 1}: \"{lines[i]}\" skipped");
    }
}
Console.WriteLine($"   {good} valid scores, average {(double)sum / good:F2}");
```

```
46
47 // 4. The same check without exceptions: TryParse returns false for bad text
48 int bad = 0;
49 foreach (string line in lines)
50 {
51     if (!int.TryParse(line, out int value))
52     {
53         bad++;
54     }
55 }
56 Console.WriteLine($"4) TryParse: {bad} bad lines");
```

Sample run

```
1) file not found: no_such_file.txt
2) IOException: no folder old_data
3) line 3: "abc" skipped
3) line 5: "" skipped
3) line 6: "70" skipped
   4 valid scores, average 83.50
4) TryParse: 3 bad lines
```

Check: $(78 + 91 + 65 + 100)/4 = 334/4 = 83.50$.

>_ 8 Semester review

The table collects the one idea per week that you should be able to explain and use in the final exam; Week 15 adds practice questions for every row.

Week	Key ideas	Typical exam task
1 Algorithms	finite, definite, input/output, effective steps; Euclid GCD, T.C. Kimlik check digits	trace GCD(48, 18): 48 mod 18 = 12, 18 mod 12 = 6, 12 mod 6 = 0 ⇒ 6
2 Flowcharts	terminal, input/output, process, decision (two labelled exits), arrows; one Start, one End	draw the flowchart of "maximum of two" or of a loop
3 Variables	`int`, `long`, `double`, `decimal` (money), `char`, `string`, `bool`; `const`; `Parse`; casts truncate: `(int)3.9` is 3	choose a type; find the missing `Parse`
4 Operators	`7 / 2` is 3, `7 / 2.0` is 3.5, `-7 % 3` is −1; precedence; `"a" + 1 + 2` is `"a12"`; `i++` vs `++i`; `&&`, `||` with short-circuit	predict the value of expressions
5 Decisions	`if/else if` (first true branch wins, order matters), `switch`, switch expression with `>= 90 =>` patterns	letter grade, leap year, boundary tests
6–7 Loops	`for` (counter), `while` (sentinel, 0 or more passes), `do-while` (menus, at least once); off-by-one, infinite loops	trace table of a loop; menu program
9–10 Arrays	index 0 . . . `Length`−1; reference type; max with index; linear search $O(n)$, binary search $O(\log n)$ on sorted data; bubble sort $O(n^2)$	one pass of bubble sort: {5, 1, 4, 2, 8} → {1, 4, 2, 5, 8}
11 Matrices	`int[,] m`, `m[r, c]`, `GetLength(0)` rows, `GetLength(1)` columns; nested loops, row and column sums	output of nested loops over {{1,2,3},{4,5,6}}: `m[1, 2]` is 6
12–13 Methods	parameters by value, arrays share the reference, `ref/out`; `return` on every path; `void` for actions; scope (CS0103, CS0136); recursion with a base case	call trace; spot the scope error; recursion trace
14 Files	`WriteAllText` replaces, `AppendAllText` adds, `ReadAllLines` gives `string[]`; streams in `using`; CSV with `Split`; `try/catch`	write a program that saves data and reads it back

>_ 9 Mini-project: Student Grade Manager

9.1 Requirements

The program manages the grades of one course. At the start it loads `students.csv` (if it exists). A menu offers: **1** add a student (name, midterm, final), **2** list all students with weighted average ($0.4 \cdot \text{midterm} + 0.6 \cdot \text{final}$) and letter grade, **3** statistics (class average, best student, number of passes), **4** search by name, **5** save, **0** exit (with automatic save). Every input is validated.

9.2 Design

Data. Three **parallel arrays** (Week 9) `names`, `midterms`, `finals` with a fixed capacity of 30, and `count`, the number of used places. Index i means the same student in all three arrays; every loop runs `i < count`. **File format.** One line per student, `name,midterm,final`, no header; names must not contain commas. **Methods.** Ten short local functions, each with one job:

Method	Kind	Idea	Week
`ReadInt(prompt, min, max)`	int, input	`TryParse` loop with range check	7, 12
`ReadName(prompt)`	string, input	not empty, no comma	12
`Average(midterm, final)`	double, pure	weighted average	4, 13
`LetterGrade(score)`	string, pure	switch expression	5
`IndexOfName(names, count, name)`	int, pure	linear search, -1 if missing	10
`AddStudent(…, count)`	int (new count)	fills the next free place	9, 12
`PrintList(…)`	void	table with alignment	6, 9
`PrintStatistics(…)`	void	sum, max with index, early return	10, 13
`SaveToFile(path, …)`	void	`StreamWriter` in `using`	14
`LoadFromFile(path, …)`	int (count)	`ReadAllLines`, `Split`, checks	14

Why do `AddStudent` and `LoadFromFile` *return* the count? The arrays are passed as references, so the methods can fill them; but `count` is an `int` and would only be changed in a copy (Week 12). Returning the new value and writing `count = AddStudent(..., count);` keeps the method free of `ref`.

9.3 Walkthrough of the code

- **Lines 4–13:** data and loading. `dataFile` is a relative path: with `dotnet run` the file lives in the project folder.
- **Lines 16–53:** the main loop is the Week 7 menu pattern `do … while (choice != 0)` with a `switch`; every case is one method call. `ReadInt` only returns $0 \ldots 5$, so no `default` is needed. After the loop the program saves automatically.
- **Input (ReadInt, ReadName):** never trust the keyboard. A comma in a name would break the CSV line, so `ReadName` refuses it.
- **Computations:** `Average`, `LetterGrade` and `IndexOfName` are pure and are reused by list, search and statistics.
- **Files:** `SaveToFile` writes only the used places (`i < count`). `LoadFromFile` accepts a line only if it has exactly three fields and both grades parse; other lines are reported and skipped. The condition `count < names.Length` protects the arrays against a file with more than 30 lines; an `IOException` (e.g. the file is open in another program) gives a message instead of a crash.

Listing 12: 14_12_GradeManager/Program.cs (complete)

```
// MCMP113 · Week 14 · Example 12 — mini-project: Student Grade Manager
// Menu (Week 7) + parallel arrays (Weeks 9-10) + methods (Weeks 12-13) + file save/load (Week 14).

// ---------- Data: parallel arrays with a fixed capacity ----------
const int Capacity = 30;                    // room for at most 30 students
string[] names = new string[Capacity];
int[] midterms = new int[Capacity];
int[] finals = new int[Capacity];
string dataFile = "students.csv";           // relative path: the project folder with dotnet run

Console.WriteLine("=== Student Grade Manager ===");
int count = LoadFromFile(dataFile, names, midterms, finals);   // number of used places
Console.WriteLine($"{count} student(s) loaded");

// ---------- Main loop: menu until the user chooses 0 ----------
int choice;
do
{
    Console.WriteLine();
    Console.WriteLine("1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit");
    choice = ReadInt("Choice: ", 0, 5);
    switch (choice)
    {
        case 1:
            count = AddStudent(names, midterms, finals, count);
            break;
        case 2:
            PrintList(names, midterms, finals, count);
            break;
        case 3:
            PrintStatistics(names, midterms, finals, count);
            break;
        case 4:
            Console.Write("Name to search: ");
            int index = IndexOfName(names, count, Console.ReadLine());
            if (index == -1)
            {
                Console.WriteLine("Not found.");
            }
            else
            {
                double avg = Average(midterms[index], finals[index]);
                Console.WriteLine($"{names[index]}: average {avg:F1}, grade {LetterGrade(avg)}");
            }
            break;
        case 5:
            SaveToFile(dataFile, names, midterms, finals, count);
            break;
    }
} while (choice != 0);

SaveToFile(dataFile, names, midterms, finals, count);       // save automatically on exit
Console.WriteLine("Goodbye!");

// ---------- Input ----------
// Asks until the user types a whole number between min and max (Week 12 ReadInt)
static int ReadInt(string prompt, int min, int max)
{
    Console.Write(prompt);
    int value;
    while (!int.TryParse(Console.ReadLine(), out value)
           || value < min || value > max)
    {
```

```
        Console.Write($"Please type a number from {min} to {max}: ");
    }
    return value;
}

// Asks for a name that is not empty and has no comma (a comma would break the CSV file)
static string ReadName(string prompt)
{
    Console.Write(prompt);
    string name = Console.ReadLine().Trim();
    while (name == "" || name.Contains(','))
    {
        Console.Write("Name must not be empty or contain a comma: ");
        name = Console.ReadLine().Trim();
    }
    return name;
}

// ---------- Computations (pure functions) ----------
// Weighted average with the MCMP113 weights: midterm 40 %, final 60 %
static double Average(int midterm, int final)
{
    return 0.4 * midterm + 0.6 * final;
}

// Letter grade on the example scale of Week 5
static string LetterGrade(double score)
{
    return score switch
    {
        >= 90 => "AA",
        >= 85 => "BA",
        >= 80 => "BB",
        >= 75 => "CB",
        >= 70 => "CC",
        >= 65 => "DC",
        >= 60 => "DD",
        _ => "FF"
    };
}

// Linear search (Week 10), ignoring upper/lower case; -1 if the name is not there
static int IndexOfName(string[] names, int count, string name)
{
    for (int i = 0; i < count; i++)
    {
        if (string.Equals(names[i], name.Trim(), StringComparison.OrdinalIgnoreCase))
        {
            return i;
        }
    }
    return -1;
}

// ---------- Actions on the arrays ----------
// Reads one student into the next free place; returns the new count
static int AddStudent(string[] names, int[] midterms, int[] finals, int count)
{
    if (count == names.Length)
    {
        Console.WriteLine("The list is full.");
        return count;
    }
    names[count] = ReadName("Name: ");
```

```
    midterms[count] = ReadInt("Midterm (0-100): ", 0, 100);
    finals[count] = ReadInt("Final (0-100): ", 0, 100);
    return count + 1;
}

// Prints all students as a table
static void PrintList(string[] names, int[] midterms, int[] finals, int count)
{
    Console.WriteLine($"{"#",2}  {"Name",-16}{"Mid",4}{"Fin",5}{"Avg",7}  Grade");
    for (int i = 0; i < count; i++)
    {
        double avg = Average(midterms[i], finals[i]);
        string letter = LetterGrade(avg);
        Console.WriteLine($"{i + 1,2}  {names[i],-16}{midterms[i],4}{finals[i],5}{avg,7:F1}  {
    letter}");
    }
}

// Prints the class average, the best student and the number of passes
static void PrintStatistics(string[] names, int[] midterms, int[] finals, int count)
{
    if (count == 0)
    {
        Console.WriteLine("No students yet.");
        return;                                     // early return (Week 13)
    }
    double sum = 0;
    int best = 0, passed = 0;
    for (int i = 0; i < count; i++)
    {
        double avg = Average(midterms[i], finals[i]);
        sum += avg;
        if (avg > Average(midterms[best], finals[best]))
        {
            best = i;                               // index of the maximum (Week 10)
        }
        if (LetterGrade(avg) != "FF")
        {
            passed++;
        }
    }
    Console.WriteLine($"Class average: {sum / count:F2}");
    Console.WriteLine($"Best student : {names[best]}");
    Console.WriteLine($"Passed       : {passed} of {count}");
}

// ---------- Files ----------
// Writes one CSV line "name,midterm,final" per student
static void SaveToFile(string path, string[] names, int[] midterms, int[] finals, int count)
{
    using (StreamWriter writer = new StreamWriter(path))
    {
        for (int i = 0; i < count; i++)
        {
            writer.WriteLine($"{names[i]},{midterms[i]},{finals[i]}");
        }
    }
    Console.WriteLine($"{count} student(s) saved to {path}");
}

// Fills the arrays from the CSV file; skips damaged lines; returns how many were read
static int LoadFromFile(string path, string[] names, int[] midterms, int[] finals)
{
    if (!File.Exists(path))
```

```
    {
        return 0;                                   // first run: nothing saved yet
    }
    string[] lines;
    try
    {
        lines = File.ReadAllLines(path);
    }
    catch (IOException)                             // e.g. the file is open in another program
    {
        Console.WriteLine($"Cannot read {path}.");
        return 0;
    }
    int count = 0;
    for (int i = 0; i < lines.Length && count < names.Length; i++)
    {
        string[] fields = lines[i].Split(',');      // "Ayse Kaya,72,64" -> 3 fields
        if (fields.Length == 3
            && int.TryParse(fields[1], out int mid)
            && int.TryParse(fields[2], out int fin))
        {
            names[count] = fields[0];
            midterms[count] = mid;
            finals[count] = fin;
            count++;
        }
        else
        {
            Console.WriteLine($"Line {i + 1} skipped: \"{lines[i]}\"");
        }
    }
    return count;
}
```

9.4 Two real runs

The first run starts without a data file, adds four students (with an invalid name and an invalid score), lists them and shows the statistics; the second run loads them again, searches, adds a fifth student and handles an invalid menu choice.

Run 1 (from the fourth student on)

```
Name: Ceren Yildiz
Midterm (0-100): 88
Final (0-100): 95

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 1
Name: Deniz, Arslan
Name must not be empty or contain a comma: Deniz Arslan
Midterm (0-100): 58
Final (0-100): 610
Please type a number from 0 to 100: 61

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 2
 #  Name            Mid  Fin   Avg  Grade
 1  Ayse Kaya        72   64  67.2  DC
 2  Burak Demir      55   66  61.6  DD
 3  Ceren Yildiz     88   95  92.2  AA
 4  Deniz Arslan     58   61  59.8  FF

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 3
Class average: 70.20
Best student : Ceren Yildiz
Passed       : 3 of 4

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 0
4 student(s) saved to students.csv
Goodbye!
```

Run 2 (complete)

```
=== Student Grade Manager ===
4 student(s) loaded

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 2
 #  Name            Mid  Fin   Avg  Grade
 1  Ayse Kaya        72   64  67.2  DC
 2  Burak Demir      55   66  61.6  DD
 3  Ceren Yildiz     88   95  92.2  AA
 4  Deniz Arslan     58   61  59.8  FF

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 4
Name to search: ceren yildiz
Ceren Yildiz: average 92.2, grade AA

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 4
Name to search: Can
Not found.

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 1
Name: Emre Sahin
Midterm (0-100): 30
Final (0-100): 41

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: x
Please type a number from 0 to 5: 3
Class average: 63.48
Best student : Ceren Yildiz
Passed       : 3 of 5

1 Add  2 List  3 Statistics  4 Search  5 Save  0 Exit
Choice: 0
5 student(s) saved to students.csv
Goodbye!
```

Check: run 1 class average $(67.2 + 61.6 + 92.2 + 59.8)/4 = 280.8/4 = 70.20$; run 2 $(280.8 + 36.6)/5 = 63.48$. Content of `students.csv` after run 2:

students.csv

```
Ayse Kaya,72,64
Burak Demir,55,66
Ceren Yildiz,88,95
Deniz Arslan,58,61
Emre Sahin,30,41
```

>_ 10 Common mistakes

File mistakes

Mistake	Effect and fix
absolute path `C:\Users\ali\data.txt`	works on one computer only; use a relative path and `Path.Combine`.
program started from another folder	`FileNotFoundException`; run it with `dotnet run` in the project folder.
`WriteAllText` instead of `AppendAllText`	the old content is lost.
`StreamWriter` without `using`	the file stays open, the last lines may be missing.
`ReadLine()` twice per pass	every second line is skipped; read once into `line`.
parsing the header row of a CSV file	`FormatException`; start at $i = 1$.
a comma inside a CSV value	an extra field; refuse commas or use another separator.
`catch (IOException)` before `catch (FileNotFoundException)`	`CS0160`; the specific type first.
empty `catch { }`	errors are hidden; print a clear message.
saving with `names.Length` instead of `count`	empty records `,0,0` are written for the free places.

>_ 11 Summary

- Variables die with the program; files on the disk survive. A relative path starts in the working directory — the project folder with `dotnet run`.
- `WriteAllText` replaces, `AppendAllText` adds, `ReadAllText` loads a string, `WriteAllLines` and `ReadAllLines` connect files with `string[]`.
- Streams (`StreamWriter`, `StreamReader`) belong in a `using` block; read with `ReadLine()` until `null`.
- CSV: one record per line; `Split(',')`, `int.Parse` of the fields, skip the header, no commas inside values.
- Persistent programs load at the start (with a default for the first run) and save after every change.
- `try/catch`: `FileNotFoundException`, `IOException`, `FormatException` — the specific type first; `TryParse` when you can test in advance.
- The Student Grade Manager joins menus, parallel arrays, methods and files: the whole semester in one program.

>_ 12 Glossary (English – Turkish)

English	Türkçe	English	Türkçe
file	dosya	stream	akış
text file	metin dosyası	to append	sonuna eklemek
binary file	ikili dosya	to overwrite	üzerine yazmak
path	yol (dosya yolu)	record / field	kayıt / alan
relative / absolute path	göreli / mutlak yol	delimiter (separator)	ayraç
working directory	çalışma dizini	header row	başlık satırı
folder (directory)	klasör (dizin)	exception	istisna (hata durumu)
to save / to load	kaydetmek / yüklemek	to throw / to catch	fırlatmak / yakalamak
persistence	kalıcılık	log file	günlük (log) dosyası
encoding	karakter kodlaması	timestamp	zaman damgası
end of file	dosya sonu	parallel arrays	paralel diziler

>_ 13 Exercises

13.1 Check yourself: predict the output

Write the exact output of Q1–Q3, then run the code to check. Each program creates its own file.

Q1

```
File.WriteAllText("a.txt", "one\n");
File.AppendAllText("a.txt", "two\n");
File.AppendAllText("a.txt", "three\n");
File.WriteAllText("a.txt", "four\n");
File.AppendAllText("a.txt", "five\n");
string[] lines = File.ReadAllLines("a.txt");
Console.WriteLine(lines.Length);
Console.WriteLine(lines[0]);
```

Q2

```
string[] rows = { "3,4", "10,2", "7,7" };
File.WriteAllLines("b.csv", rows);
int total = 0;
foreach (string row in File.ReadAllLines("b.csv
    "))
{
    string[] f = row.Split(',');
    total += int.Parse(f[0]) * int.Parse(f[1]);
}
Console.WriteLine(total);
```

Q3

```
string[] data = { "10", "x", "20" };
File.WriteAllLines("c.txt", data);
int sum = 0;
foreach (string s in File.ReadAllLines("c.txt")
    )
{
    try
    {
        sum += int.Parse(s);
        Console.Write("ok ");
    }
    catch (FormatException)
    {
        Console.Write("bad ");
    }
}
Console.WriteLine(sum);
```

13.2 In-class exercises (pairs)

E1. ⋆ **Squares file.** Write the lines `n,n²` (e.g. `3,9`) for $n = 1 \ldots 10$ to `squares.csv` with a `StreamWriter`. Read the file back and print the sum of the squares.

E2. ⋆⋆ **Numbered copy.** Copy `proverbs.txt` (create it as in Example 7 if it does not exist) to `numbered.txt` with line numbers `1: …`, skipping empty lines. Print how many lines were written.

E3. ⋆⋆ **Shopping CSV.** Create `items.csv` with the header `item,price,qty` and three items. Print

every line total and the grand total, using `decimal` and `F2`.

E4. ⋆ **Find the bug.** Predict the output of the first loop, then fix it.

```
File.WriteAllLines("days.txt", new string[] { "Mon", "Tue", "Wed", "Thu", "Fri" });
using (StreamReader reader = new StreamReader("days.txt"))
{
    while (reader.ReadLine() != null)
    {
        Console.WriteLine(reader.ReadLine());
    }
}
```

13.3 Homework

H1. ⋆⋆ **Sort the students.** Add menu option `6 Sort` to the Grade Manager: bubble sort (Week 10) by average, highest first. All three arrays must be swapped together.

H2. ⋆⋆ **Word frequency.** Write `static int CountWord(string path, string word)` that counts how often a whole word appears in a text file, ignoring upper/lower case and the punctuation `. , ! ?`.

H3. ⋆⋆⋆ **Matrix to CSV and back.** Save an `int[3, 4]` matrix (Week 11) as CSV, one row per line, and load it back into a new `int[,]`. Print the file and the loaded matrix.

H4. ⋆ **Diary.** Append a typed sentence with a timestamp to `diary.txt`, then print the last three entries (or fewer, if the file is shorter).

13.4 Extra practice

X1. ⋆ A student copies the compiled `GradeManager.dll` to the desktop and starts it there. Where does the program look for `students.csv`, and what does it report?

X2. ⋆ Which of these calls keep the existing content of `a.txt`? (a) `File.WriteAllText("a.txt", s)` (b) `File.AppendAllText("a.txt", s)` (c) `new StreamWriter("a.txt")` (d) `new StreamWriter("a.txt", true)` (e) `File.ReadAllLines("a.txt")`

X3. ⋆⋆ In Example 4, which line(s) of `grades.csv` cause a `FormatException` if the loop starts at $i = 0$, and what would happen if the line `Can Er,abc,70` were added at the end?

>_ 14 References

- Microsoft Learn — File and stream I/O: https://learn.microsoft.com/en-us/dotnet/standard/io/
- Microsoft Learn — How to write text to a file: https://learn.microsoft.com/en-us/dotnet/standard/io/how-to-write-text-to-a-file
- Microsoft Learn — How to read text from a file: https://learn.microsoft.com/en-us/dotnet/standard/io/how-to-read-text-from-a-file
- Microsoft Learn — File paths on Windows systems: https://learn.microsoft.com/en-us/dotnet/standard/io/file-path-formats
- Microsoft Learn — File class (API): https://learn.microsoft.com/en-us/dotnet/api/system.io.file
- Microsoft Learn — The using statement: https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/using
- Microsoft Learn — Exception-handling statements (try, catch): https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/exception-handling-statements
- Microsoft Learn — String.Split: https://learn.microsoft.com/en-us/dotnet/csharp/how-to/parse-strings-using-split
- Microsoft Learn — Compiler error CS0160: https://learn.microsoft.com/en-us/dotnet/csharp/misc/cs0160